

0.58. Plus large que haut : rectangle

`rectangle(x, y, tuile, largeur, hauteur, sens, vitesse, tours)` : un carré avec deux tailles, une pour X et une pour Y.

1. Ce qu'il faut comprendre

Un rectangle, c'est un carré qui n'a pas la même taille sur les deux axes. Au lieu d'une `taille`, il en a **deux** : la **largeur** (sur X) et la **hauteur** (sur Y).

Ce qui est nouveau ici : `rectangle(x, y, tuile, largeur, hauteur, sens, vitesse, tours)`, **huit** réglages : ceux de `carre`, avec `largeur` et `hauteur` à la place de `taille`.

- `largeur` et `hauteur` se comptent comme la taille d'un carré : la distance du centre au bord. La règle $2 \times n + 1$ vaut pour chacune : largeur 4 donne **9** cases de large, hauteur 2 donne **5** cases de haut, un rectangle de **9 × 5**.
- Avec la même largeur et la même hauteur, `rectangle` fait **un carré** : `rectangle(10, 8, A, 2, 2, ...)` fait le même tour que `carre(10, 8, A, 2, ...)`.

Le trajet : du centre, en diagonale, puis tout droit jusqu'au coin en haut à gauche ; le tour ; puis le même chemin à l'envers.

Les plus grandes tailles : largeur 9 (19 colonnes) et hauteur 8 (17 lignes).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2301 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Un rectangle autour de la case (10, 8) :
    //
    // rectangle(10, 8, ALPHABET[0], 4, 2, 1, 250, 1);
    //
    //      | | |           | | | | |
    //      | | |           | | | +-- tours   : 1 tour
    //      | | |           | | | +----- vitesse : 250 ms par pas
    //      | | |           | | +----- sens    : 1, comme les aiguilles
    //      | | |           | +----- hauteur : 2 → 2 × 2 + 1 = 5 cases de haut
    //      | | |           +----- largeur  : 4 → 2 × 4 + 1 = 9 cases de large
    //      | | +----- tuile   : la lettre A
    //      +----- x, y      : le centre (10, 8)
    //
    //      colonne :   6  7  8  9 10 11 12 13 14
    //      ligne 6 :   → → → → → → → ↓      les côtés du haut et du bas :
    //      ligne 7 :   ↑                ↓      8 pas (2 × largeur)
    //      ligne 8 :   ↑          A        ↓
    //      ligne 9 :   ↑                ↓      les côtés de gauche et de droite :
    //      ligne 10 :  ↑ ← ← ← ← ← ← ←      4 pas (2 × hauteur)
    //
    // Le coin de départ est en haut à gauche, (6, 6) : le A y va du centre en
    // diagonale (2 pas), puis tout droit (2 pas), et en revient de même.
    rectangle(10, 8, ALPHABET[0], 4, 2, 1, 250, 1);

    while (true) {
        image();          // le tour est fini : le A reste au centre
    }
}
```

3. Ce qui se passe, étape par étape

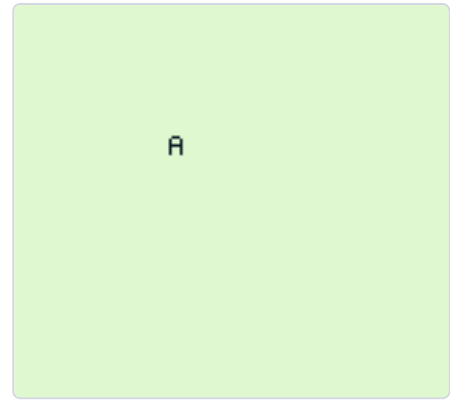
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



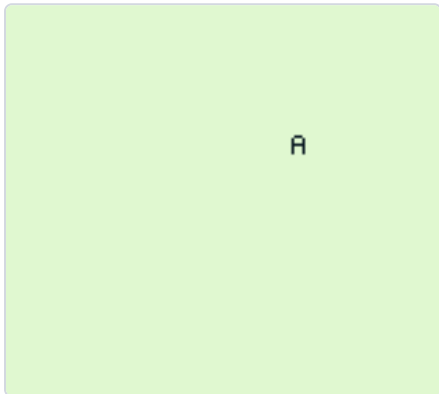
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui fait le tour d'un rectangle de 9 cases de large et 5 de haut autour de (10, 8), puis revient au centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les quatre coins : (6, 6), (14, 6), (14, 10), (6, 10)

✓ 9 de large, 5 de haut : jamais plus loin

✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.58.1. Plus large que haut : rectangle — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++